Court-circuit : le mémo

Prénom : ..... Classe : .....

Ton robot colorie les cases où il passe ; une case adverse sur laquelle tu passes devient la tienne. Après 30 s, le plus de cases gagne. En ligne : **babash.github.io/court-circuit**

Les actions (une à la fois)

`avancer()`	avance d'une case et la colorie
`droite()` `gauche()`	tourne d'un quart de tour, sur place
`eclair()`	éclair sur 3 cases devant : l'adversaire touché est figé 2 s et les cases autour de lui deviennent les tiennes
`attendre()`	ne fait rien

Les capteurs (des questions)

`libre()`	Vrai si la case devant est libre ; aussi `libre("droite")`, `libre("gauche")`
`couleur()`	`"vide"`, `"moi"`, `"adversaire"` ou `"mur"`
`cible()`	Vrai si un éclair toucherait l'adversaire
`distance()`	le nombre de cases jusqu'à l'adversaire

L'essentiel

Boucle bornée : répéter un nombre de fois fixé.

```
for i in range(4):
    avancer()
```

Boucle non bornée : répéter tant qu'une condition est vraie.

```
while True:
    if libre():
        avancer()
    else:
        droite()
```

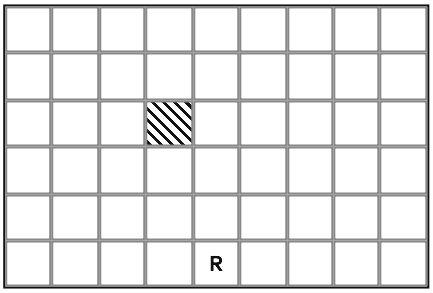
> **Attention** : une ligne qui commence par `for`, `while`, `if`, `elif` ou `else` finit par « : », et les lignes de son bloc sont décalées de 4 espaces.

Vitesse : chaque ligne recopiée ralentit ton robot. Une boucle évite de recopier.

Exercice 1 — Lire un code

Le robot part de la case R, vers le haut. Colorier les cases qu'il visite avec ce code, puis compter les cases coloriées.

```
for i in range(3):
    avancer()
droite()
for i in range(2):
    avancer()
```



Nombre de cases (départ compris) : .....

Exercice 2 — Recopier ou répéter ?

Écrire avec une boucle `for` un code qui fait la même chose que : `avancer()` `avancer()` `avancer()` `droite()` `avancer()` `avancer()` `avancer()` `droite()`.

Exercice 3 — Choisir sa condition

Compléter pour que le robot avance sur les cases qui ne sont pas à lui, et tourne à droite sinon :

```
while True:
    if libre() …… couleur() …… "moi":
        avancer()
    else:
        ……
```

Je sais…

☐ écrire une boucle `for` pour répéter un nombre de fois fixé ;
☐ utiliser `while` pour répéter tant qu'une condition est vraie ;
☐ écrire `if` / `elif` / `else` avec les deux-points et le décalage ;
☐ combiner deux conditions avec `and`, `or` ou `not`, et comparer avec `==`, `!=`, `<=`.

Court-circuit · Babash - NSI Tani Malandi · licence CC BY-SA 4.0 (documents) et MIT (code) · Code source sur GitHub : github.com/babash/court-circuit